STAYKARO PRIVATE LIMITED

AI CALLER AGENT

Infrastructure & Operations Guide

Docker, deployment, monitoring, and disaster recovery for V1

Version 1.1
Primary owner: Nihal (Execution Plan §D)
Companion to the Architecture Specification, §29-33

Table of Contents

1. Docker & Docker Compose 3

2. VPS & Production Stack Layout 4

3. Nginx (Reverse Proxy) 5

4. CI/CD 5

5. Monitoring, Alerts & Logging 6

6. Backups & Disaster Recovery 6

6.1 Disaster recovery scenarios 7

7. Deployment, Rollback & Graceful Shutdown 7

7.1 Rollback 8

8. Scaling 8

9. Production Checklist 8

1. Docker & Docker Compose

CONCEPT · Docker

WHAT IT IS

Docker packages an application together with everything it needs to run -- the exact Python version, libraries, system dependencies -- into a single, portable unit called a container, so it runs the same way on any machine.

REAL-WORLD ANALOGY

It's a shipping container, literally. Before standardized shipping containers, loading a ship was a custom, slow negotiation for every different type of cargo. A shipping container doesn't care what's inside -- any crane, any ship, any truck can move it identically. Docker does that for software: your laptop, the CI server, and the production VPS all run the exact same container, so "works on my machine" stops being a real category of bug.

WHY STAYKARO USES IT

Shivashree, Nishkala, and Nihal are on different machines, and the production VPS is a fourth, different environment again. Docker is what guarantees the voice-gateway that passed tests locally behaves identically once deployed.

TECHNICAL IMPLEMENTATION

Every service (Developer Handbook §3) has its own Dockerfile; `docker-compose.yml` wires them together for local development and for the production VPS alike.

CONCEPT · Docker Compose

WHAT IT IS

Docker Compose starts and connects multiple containers together with a single command, using one configuration file to describe how they relate.

REAL-WORLD ANALOGY

If Docker is a shipping container, Compose is the entire loading manifest for a ship carrying ten different containers that need to sit in a specific arrangement relative to each other.

WHY STAYKARO USES IT

This platform is not one program -- it's eight or nine cooperating services (voice-gateway, api, worker, redis, nginx, integration-service, monitoring, and conditionally n8n). Compose is what starts all of them, in the right order, networked together, with one command.

TECHNICAL IMPLEMENTATION

`docker compose up` reads `docker-compose.yml` (Architecture Spec §29) and brings up every service with its restart policy and health check.

```yaml
services:
  voice-gateway:
    build: ./services/voice-gateway
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 15s
      timeout: 5s
      retries: 3
    depends_on:
      redis:
        condition: service_healthy

  redis:
    image: redis:7-alpine
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
```

Figure 1.1 -- Representative docker-compose.yml excerpt, with restart policy and health checks

restart: unless-stopped IS NOT A COMPLETE AVAILABILITY STRATEGY

It recovers from a crashed process. It does nothing for a bad deploy (§7), a full host failure, or Redis losing state mid-call. Host-level monitoring (§4) and the deployment-draining sequence (§7) exist precisely because this alone isn't enough.

2. VPS & Production Stack Layout

CONCEPT · VPS (Virtual Private Server)

WHAT IT IS

A VPS is a persistent, always-on server you rent, with a fixed amount of CPU/RAM/disk reserved for you, as opposed to "serverless" hosting that spins compute up and down on demand.

REAL-WORLD ANALOGY

Renting an apartment (VPS) versus paying for a hotel room only when you happen to need one (serverless). An apartment is always there, always yours, always has your furniture in it -- exactly what you want if you need somewhere to permanently live, not just visit occasionally.

WHY STAYKARO USES IT

The voice gateway holds long-lived WebSocket connections and Redis holds in-memory state that must persist across many requests over the life of a call. Serverless platforms are built to spin down when idle -- exactly wrong for a process that needs to stay up, mid-conversation, indefinitely. This is the same lesson the LMS project's backend already taught: anything with persistent workers or WebSockets needs a real, always-on host, not Vercel-style serverless.

TECHNICAL IMPLEMENTATION

The voice-gateway, api, worker, redis, and integration-service all run in Docker Compose on one 24/7 VPS. The two Next.js dashboards, which are stateless, run on Vercel instead.

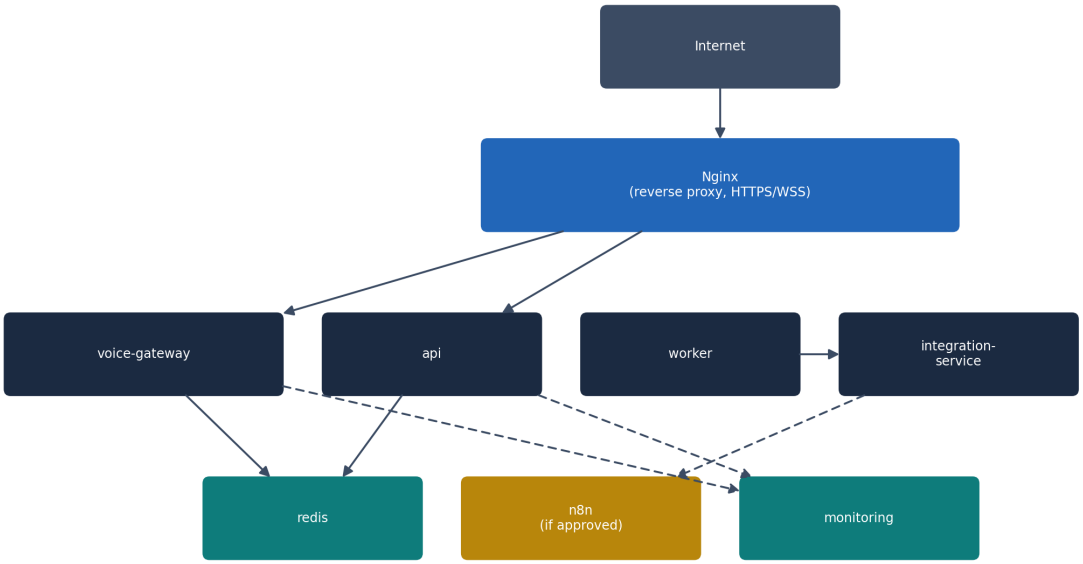

Figure 2.1 -- Production VPS service layout. Solid lines are always-on connections; dashed lines are async/best-effort.

3. Nginx (Reverse Proxy)

CONCEPT · Reverse proxy

WHAT IT IS

A reverse proxy sits in front of your actual application servers and is the only thing directly exposed to the internet -- it receives every request and forwards it to the right internal service.

REAL-WORLD ANALOGY

It's a hotel's front desk. Guests don't wander the building looking for their room; they tell the front desk what they need, and the front desk directs them -- while also being the one place that can enforce "no unregistered guests past this point" (HTTPS, rate limiting) for the whole building at once.

WHY STAYKARO USES IT

The platform has multiple internal services (api, voice-gateway) that shouldn't each individually manage TLS certificates, rate limiting, and public exposure. Nginx does that once, centrally, and forwards traffic internally over the private Docker network.

TECHNICAL IMPLEMENTATION

Nginx terminates HTTPS/WSS, routes `/api/*` to the api service and the voice WebSocket path to voice-gateway, and applies coarse per-IP rate limiting (API Spec §1.3) before a request ever reaches application code.

4. CI/CD

■ QUICK REMINDER — CI/CD

Continuous Integration / Continuous Deployment -- the automated pipeline that tests every change and, once approved, ships it to production without manual, error-prone steps. *(Full explanation: Developer Handbook.)* Full detail and the exact pipeline stages are in the Developer Handbook, §11 -- summarized here as the thing that runs before any of the deployment steps in §7 below happen.

5. Monitoring, Alerts & Logging

CONCEPT · Monitoring vs. alerting vs. logging -- three different jobs

WHAT IT IS

Logging records what happened (Developer Handbook §8). Monitoring continuously measures the system's current state -- latency, error rate, active calls. Alerting is the rule that turns "monitoring noticed something bad" into a human actually finding out, right now, without needing to be staring at a dashboard.

REAL-WORLD ANALOGY

A car has a fuel gauge (monitoring, always visible), a trip log (logging, reviewed later), and a warning light plus chime when fuel is nearly empty (alerting, demands attention now). All three matter, and they fail differently: a fuel gauge with no warning light means you find out you're empty by stalling.

WHY STAYKARO USES IT

A degraded Groq API or a Redis outage needs someone paged within minutes, not discovered the next morning in a log review -- especially for a real-time system where every minute of an undetected outage is active calls failing.

TECHNICAL IMPLEMENTATION

Metrics tracked per Architecture Spec §32: active calls, per-stage latency, provider failures, error rate, database/Redis latency, cost per call. Alerts configured for: elevated error rate, latency breaching target, provider outage, database/Redis unavailability.

Signal	Example threshold	Action
Total response latency (p95)	> 1.5s sustained for 5 minutes	Page on-call
Provider failure rate (any provider)	> 5% of calls in 10 minutes	Page on-call, check provider status page
Database/Redis unreachable	Any occurrence	Immediate page
Failed/dropped call rate	> 2% sustained	Page on-call
Disk usage on VPS	> 85%	Alert (non-paging) to team channel

6. Backups & Disaster Recovery

■ QUICK REMINDER — Backup & restoration testing

A saved copy of the database, and the (mandatory, tested) process of actually using it to recover. *(Full explanation: Developer Handbook.)* Full explanation and reasoning in the Database Design document, §10. The operational commitment here: restoration is tested before production launch, and re-tested on a recurring schedule afterward -- not a one-time box-check.

6.1 Disaster recovery scenarios

Scenario	Recovery approach
Single container crashes	Docker restart policy recovers automatically (§1)
Bad deploy breaks a service	Roll back (§8) to the last known-good image
Redis loses all data	Active calls fail safely and end (Architecture Spec §18) -- new calls start clean; no permanent data is lost since Redis never held the only copy of anything permanent
PostgreSQL corrupted or lost	Restore from the most recent backup + WAL replay (Database Design §10); this is the scenario the mandatory restoration test proves is actually survivable
Entire VPS host fails	Provision a new VPS, redeploy via CI/CD and Docker Compose, restore PostgreSQL from backup -- accepted as a real-outage scenario for V1 (Architecture Spec §39), not instant failover

7. Deployment, Rollback & Graceful Shutdown

CONCEPT · Graceful shutdown

WHAT IT IS

Graceful shutdown means a process finishes what it's currently doing before it stops, rather than being killed mid-task.

REAL-WORLD ANALOGY

A restaurant putting up a "no new customers, but we'll finish serving everyone already seated" sign at closing time, instead of turning off the lights and locking the door on people mid-meal.

WHY STAYKARO USES IT

A deploy that just kills and replaces the voice-gateway container instantly would hang up every active caller. Architecture Spec §30 makes graceful shutdown a hard requirement, not a nice-to-have, for exactly this reason.

TECHNICAL IMPLEMENTATION

The voice gateway listens for a shutdown signal, immediately stops accepting new calls, lets active calls finish naturally, and only then allows the container to actually stop.

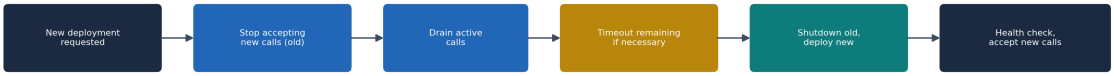

Figure 7.1 -- The deployment sequence. No step here is skippable for a live system.

7.1 Rollback

If a deploy is bad -- elevated error rate, failed health checks -- the response is to redeploy the previous known-good container image through the same CI/CD pipeline (§4), not to hand-patch the running container. Every deployed image is tagged and kept, specifically so "go back to what worked" is a single command, not an investigation.

ROLLBACK TRIGGER

Any of: health check failing post-deploy, error rate elevated beyond the §5 threshold within 10 minutes of deploy, or a manual call from whoever is on-call. Don't wait for certainty -- rolling back a false alarm costs minutes; leaving a genuinely bad deploy running costs live calls.

8. Scaling

CONCEPT · Scaling -- and why V1 deliberately doesn't do much of it yet

WHAT IT IS

Scaling means adding capacity (more servers, more instances) to handle more simultaneous load.

REAL-WORLD ANALOGY

V1 is buying one apartment, not building a hotel chain. You don't build a hotel chain's booking system before you've confirmed people actually want to stay in the one apartment.

WHY STAYKARO USES IT

Architecture Spec §39 explicitly treats the single-VPS setup as an accepted V1 tradeoff -- the highest-leverage thing to prove first is that the product works and clients want it, not that it survives load nobody has generated yet.

TECHNICAL IMPLEMENTATION

Concurrency capacity is measured, not assumed (Execution Plan §K, load testing). Multi-VPS/load-balanced scaling (Architecture Spec §43 roadmap) is deliberately deferred until real traffic actually justifies the added operational complexity.

9. Production Checklist

Reproduced in full in the Team Execution Plan, §P, and the Architecture Specification, §41 -- the infrastructure-specific subset relevant to this document:

- Docker Compose brings up every service healthy on a clean machine (§1)
- Nginx enforces HTTPS/WSS and rate limiting in front of every public route (§3)

- Monitoring and alerting are live for every critical service, with thresholds matching §5
- A backup has been restored to a clean environment and verified, not just scripted (§6)
- Deployment draining has been verified against a real active call, not just reviewed on paper (§7)
- Rollback has been exercised at least once before launch, not only documented (§7.1)
- Concurrency capacity has been measured under load, not assumed (§8)